
# IT 309 SOFTWARE ENGINEERING

## PROJECT DOCUMENTATION

FITSCHEDULE

Prepared by:
**Sarah Spahić**
**Adi Džumhur**

Proposed to:
**Nermina Durmić, Assist. Prof. Dr.**
**Ajla Korman, Teaching Assistant**
**Amila Čaušević, Teaching Assistant**

05. April, 2026

## 1. Project Overview (Milestone 1)

FitSchedule[1] is a web-based application designed for gyms and fitness centres that enables clients to search for and book personal training sessions with available trainers. The platform digitises and automates the gym scheduling process, replacing manual phone or reception-based booking with a seamless digital experience.

The application consists of three user roles: clients who browse and book training sessions, trainers who manage their availability and track their sessions, and administrators who oversee the entire platform. A client can search for available trainers by selecting a desired date and time slot, view trainer profiles, and reserve a session directly through the app. The trainer receives a notification, sees the booking in their weekly schedule, and can mark the session as completed once it is done. After a completed session, the client is prompted to leave a rating and review for the trainer.

The system is built as a web application with a separate frontend and backend. The frontend is developed using React, ensuring a responsive and intuitive user interface across all devices. The backend is built with Java Spring Boot, exposing a RESTful API that the frontend communicates with. Data is stored in a MySQL relational database, which is well-suited for the structured nature of bookings, users, trainers, and time slots.

[1] GitHub repository: https://github.com/sparah27/FitSchedule

## 2. High-Level Plan

The FitSchedule project will be developed using an Agile methodology, chosen for its flexibility and ability to accommodate changing requirements throughout the development process. Rather than defining every detail upfront, the team will work in iterative two-week sprints, continuously delivering working software and refining the product based on progress and feedback.

The project is organised into two major releases. The first release covers the core client-facing functionality, specifically the features that allow gym members to register, search for trainers, and book training sessions. This release establishes the foundation of the application and delivers a functional, usable product for the client role. The second release builds upon this foundation by introducing the trainer portal, the admin panel, and additional features such as reviews, ratings, and session management. By the end of the second release, the team will have implemented full functionality across all three user roles and deployed the application publicly.

This incremental approach ensures that the team delivers the most critical features first, identifies risks early, and distributes the workload evenly across the development timeline.

### 2.1 Product Roadmap

| Release | Timeframe | Theme | Key Features |
|---|---|---|---|
| Release 1 (v1.0) | Weeks 1–4 / By May 3, 2026 | Core Booking Platform (Client Side) | User registration & login, Browse & search trainers by date/time, View trainer profiles & available slots, Book a session, View and cancel own bookings, Basic in-app notifications |
| Release 2 (v2.0) | Weeks 5–8 / By June 7, 2026 | Full Platform (Trainer & Admin) | Trainer availability management, Trainer weekly schedule view, Mark sessions as completed, Client reviews & ratings, Admin dashboard & statistics, Admin trainer management, Deployment |

## 2.2 Release Plan

| Sprint | Duration | Goals | User Stories |
|---|---|---|---|
| Sprint 1 | Weeks 1–2 | Project setup, database design, authentication backend, basic frontend scaffolding | US-01, US-12 |
| Sprint 2 | Weeks 3–4 | Trainer listing & filtering, trainer profile view, availability display, booking creation & cancellation, My Bookings page, notifications | US-02, US-03, US-04, US-05, US-06, US-07, US-08, US-09, US-13, US-14 |
| Sprint 3 | Weeks 5–6 | Trainer portal: availability management, weekly schedule view, session completion, block slots, profile edit, notifications | US-15, US-16, US-17, US-18, US-19, US-20, US-21 |
| Sprint 4 | Weeks 7–8 | Admin panel: trainer management, all-bookings view, deactivation, statistics dashboard, reviews, ratings, and deployment | US-10, US-11, US-22, US-23, US-24, US-25 |

# 3. User Stories

The following section presents the functional and non-functional requirements for FitSchedule in the form of user stories. User stories are a core Agile practice that describe system functionality from the perspective of the end user, keeping the focus on what the user needs rather than technical implementation details. Each user story follows the format "As a [actor], I want to [goal] so that [benefit]" and includes acceptance criteria that define when the story is considered complete and ready for review.

FitSchedule has three distinct user roles, each with their own set of responsibilities and interactions with the system. Clients are gym members who use the platform to find and book personal training sessions. Trainers are gym staff who manage their availability, track their upcoming sessions, and communicate with clients through the platform. Admins are gym managers who oversee all activity on the platform, manage trainer accounts, and monitor overall gym performance through a dedicated dashboard.

## 3.1 Functional Requirements

### 3.1.1 Client

**US-01:** As a client, I want to register an account so that I can access the FitSchedule platform.

*Acceptance criteria:* Given I am on the registration page, when I enter a valid email, password, name, and phone number and click Register, then my account is created and I am redirected to the dashboard.

**US-02:** As a client, I want to view a list of all available trainers so that I can choose the one that suits me best.
Acceptance criteria: Given I am logged in, when I navigate to the Trainers page, then I see a list of trainers with their name, photo, specialization, and rating.

**US-03**: As a client, I want to filter trainers by specialization so that I can find a trainer relevant to my fitness goals.
*Acceptance criteria:* Given I am on the Trainers page, when I apply a specialization filter (e.g. strength, yoga), then only trainers with that specialization are displayed.

**US-04:** As a client, I want to view a trainer's profile so that I can learn about their experience and available time slots.
*Acceptance criteria:* Given I click on a trainer, when their profile page loads, then I can see their bio, certifications, specializations, ratings, and a calendar of available slots.

**US-05:** As a client, I want to search for available trainers by date and time so that I can find who is free when I want to train.
*Acceptance criteria:* Given I am on the search page, when I select a date and time range, then the system displays all trainers with at least one free slot in that period.

**US-06:** As a client, I want to book a session with a trainer by selecting a time slot so that I can reserve my training session.
*Acceptance criteria:* Given I have selected a trainer and an available slot, when I confirm the booking, then the slot is reserved, the trainer is notified, and I see a booking confirmation.

**US-07:** As a client, I want to view all my upcoming bookings so that I can plan my schedule.
*Acceptance criteria:* Given I am logged in, when I navigate to My Bookings, then I see a list of all upcoming sessions with trainer name, date, time, and status.

**US-08:** As a client, I want to cancel a booking so that I can free up the slot if my plans change.
*Acceptance criteria:* Given I have an upcoming booking, when I click Cancel and confirm, then the booking is cancelled, the slot becomes available again, and the trainer is notified.

**US-09:** As a client, I want to receive a notification when my booking is confirmed or cancelled so that I am always informed.
*Acceptance criteria:* Given a booking status change, when the change is saved, then I receive an in-app notification within 1 minute.

**US-10:** As a client, I want to leave a rating and review for a trainer after a completed session so that I can share my experience.
*Acceptance criteria:* Given a session is marked as completed, when I navigate to that booking and submit a rating (1-5 stars) and an optional comment, then the review is saved and visible on the trainer's profile.

**US-11:** As a client, I want to view my past sessions so that I can track my training history.
*Acceptance criteria:* Given I am on My Bookings, when I switch to the Past tab, then I see all completed and cancelled sessions with date, trainer, and status.

**US-12:** As a client, I want to update my profile information so that my contact details and preferences are up to date.
*Acceptance criteria:* Given I am on my Profile page, when I edit my name, phone, or profile photo and save, then the changes are persisted and displayed immediately.

**US-13:** As a client, I want to add a trainer to my favourites so that I can quickly book them again.
*Acceptance criteria:* Given I am viewing a trainer's profile, when I click the Favourite button, then the trainer is added to my favourites list accessible from my dashboard.

**US-14:** As a client, I want to see the gym's weekly class schedule so that I can plan group sessions as well.
*Acceptance criteria:* Given I am on the Schedule page, when the page loads, then I see a weekly calendar view with all group classes, their times, and available spots.

### 3.1.2 Trainer

**US-15:** As a trainer, I want to set my weekly availability so that clients can only book me during my working hours.
*Acceptance criteria:* Given I am on my Availability settings, when I define my available days and time ranges and save, then clients can only select those slots when booking me.

**US-16:** As a trainer, I want to view my full schedule for the week so that I know all my upcoming sessions.
*Acceptance criteria:* Given I am logged in as a trainer, when I open My Schedule, then I see a weekly calendar with all booked sessions showing client name, time, and session type.

**US-17:** As a trainer, I want to receive a notification when a client books or cancels a session with me so that I can adjust my plans.
*Acceptance criteria:* Given a client books or cancels a session, when the action is completed, then I receive an in-app notification within 1 minute.

**US-18:** As a trainer, I want to mark a session as completed so that the client can leave a review and the system records it.
*Acceptance criteria:* Given a session has passed, when I click Mark as Completed on that booking, then the session status updates and the client is prompted to leave a review.

**US-19:** As a trainer, I want to view all reviews left by my clients so that I can monitor my performance and reputation.
*Acceptance criteria:* Given I am on my Profile page, when I scroll to the Reviews section, then I see all client reviews with star ratings, comments, and dates.

**US-20:** As a trainer, I want to block off specific time slots as unavailable so that I can manage personal appointments.
*Acceptance criteria:* Given I am on my Availability calendar, when I mark a specific slot as blocked and save, then that slot no longer appears as available to clients.

**US-21:** As a trainer, I want to update my profile and specializations so that clients have accurate information about me.
*Acceptance criteria:* Given I am on my Profile edit page, when I update my bio, photo, or specializations and save, then the changes are reflected on my public profile immediately.

### 3.1.3 Admin

**US-22:** As an admin, I want to add and manage trainer accounts so that the gym's trainer roster is always up to date.
*Acceptance criteria:* Given I am in the Admin panel, when I create a new trainer account with name, email, and specializations, then the trainer receives an activation email and appears in the trainer list.

**US-23:** As an admin, I want to view all bookings across the gym so that I have full visibility of gym activity.
*Acceptance criteria:* Given I am on the Admin Bookings page, when the page loads, then I see all bookings with client name, trainer name, date, time, and status, with options to filter by date or status.

**US-24:** As an admin, I want to deactivate a trainer account so that former trainers no longer appear as available.
*Acceptance criteria:* Given I am viewing a trainer's account, when I click Deactivate and confirm, then the trainer is hidden from client searches and cannot log in.

**US-25:** As an admin, I want to view a dashboard with key gym statistics so that I can monitor gym performance.
*Acceptance criteria:* Given I am on the Admin Dashboard, when the page loads, then I see total bookings, active trainers, active clients, and cancellation rate for the current month.

## 3.2 Non-Functional Requirements

**NF-01: Performance**
As a user, I want the application to load pages within 2 seconds so that my experience is smooth and efficient.

*Acceptance criteria:* Given any page is requested, when the server responds, then the full page content must be rendered within 2 seconds under normal network conditions.

**NF-02: Usability**
As a user, I want the application to be responsive and work on mobile, tablet, and desktop so that I can use it on any device.

*Acceptance criteria:* Given I access the app on a device with any screen width from 320px to 1920px, when I navigate the app, then all elements are properly displayed and functional with minimal horizontal scrolling.

**NF-03: Security**
As a user, I want my personal data and bookings to be protected so that my privacy and account security are ensured.

*Acceptance criteria:* Given a user submits credentials, when authentication is processed, then passwords are hashed using bcrypt, all API communication occurs over HTTPS, and JWT tokens expire within 24 hours.

## 4. UML Diagrams

## 4.1 Activity Diagrams

The following activity diagrams illustrate the workflows and decision points for three core functionalities of the FitSchedule application. The first diagram covers the process of searching for and booking a training session from the client's perspective. The second diagram focuses on how a trainer defines and manages their weekly availability. The third diagram shows how an admin adds a new trainer to the platform, including account creation and email activation.

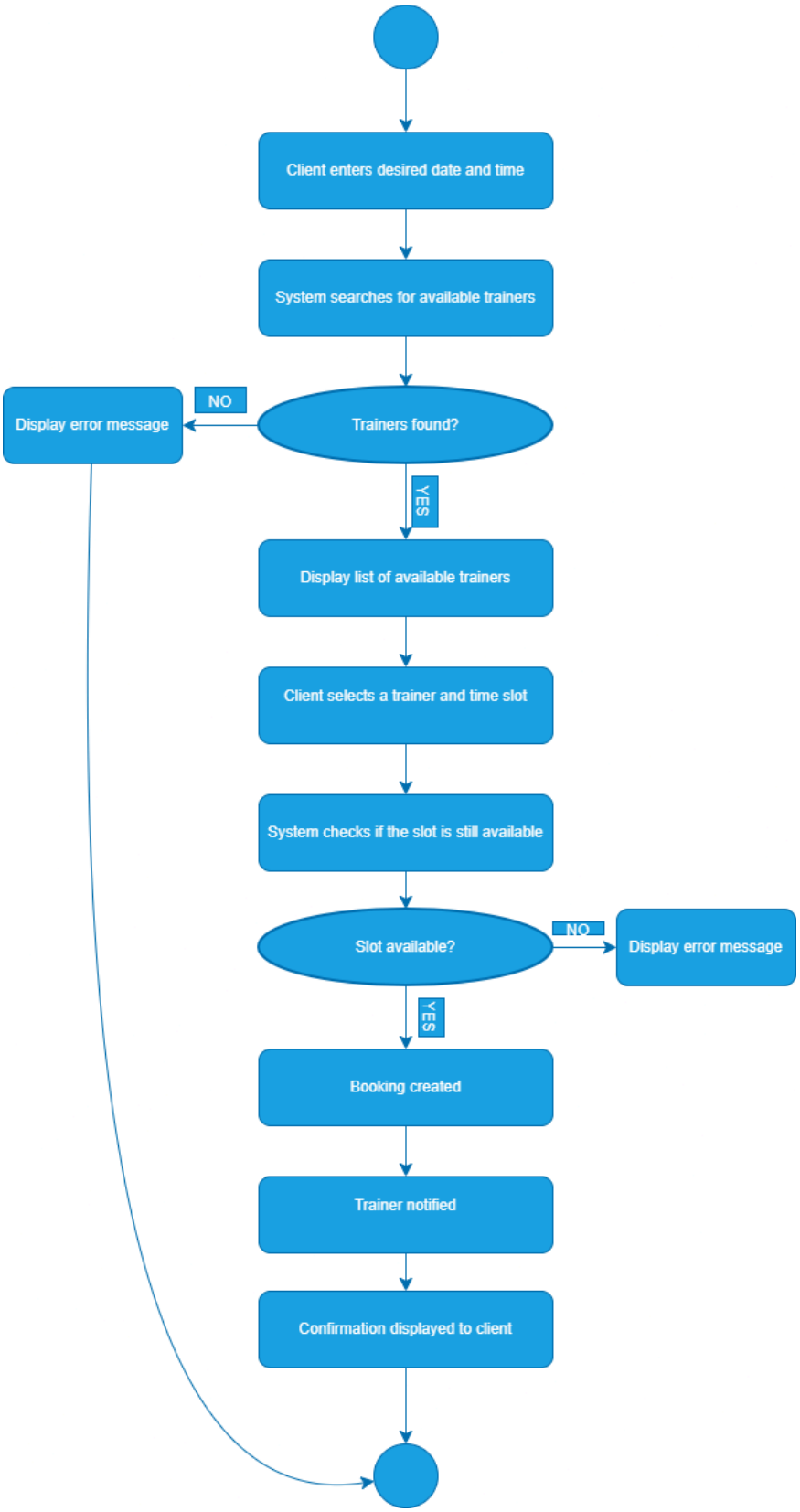



Figure 1: Activity Diagram - Search & Book a Trainer

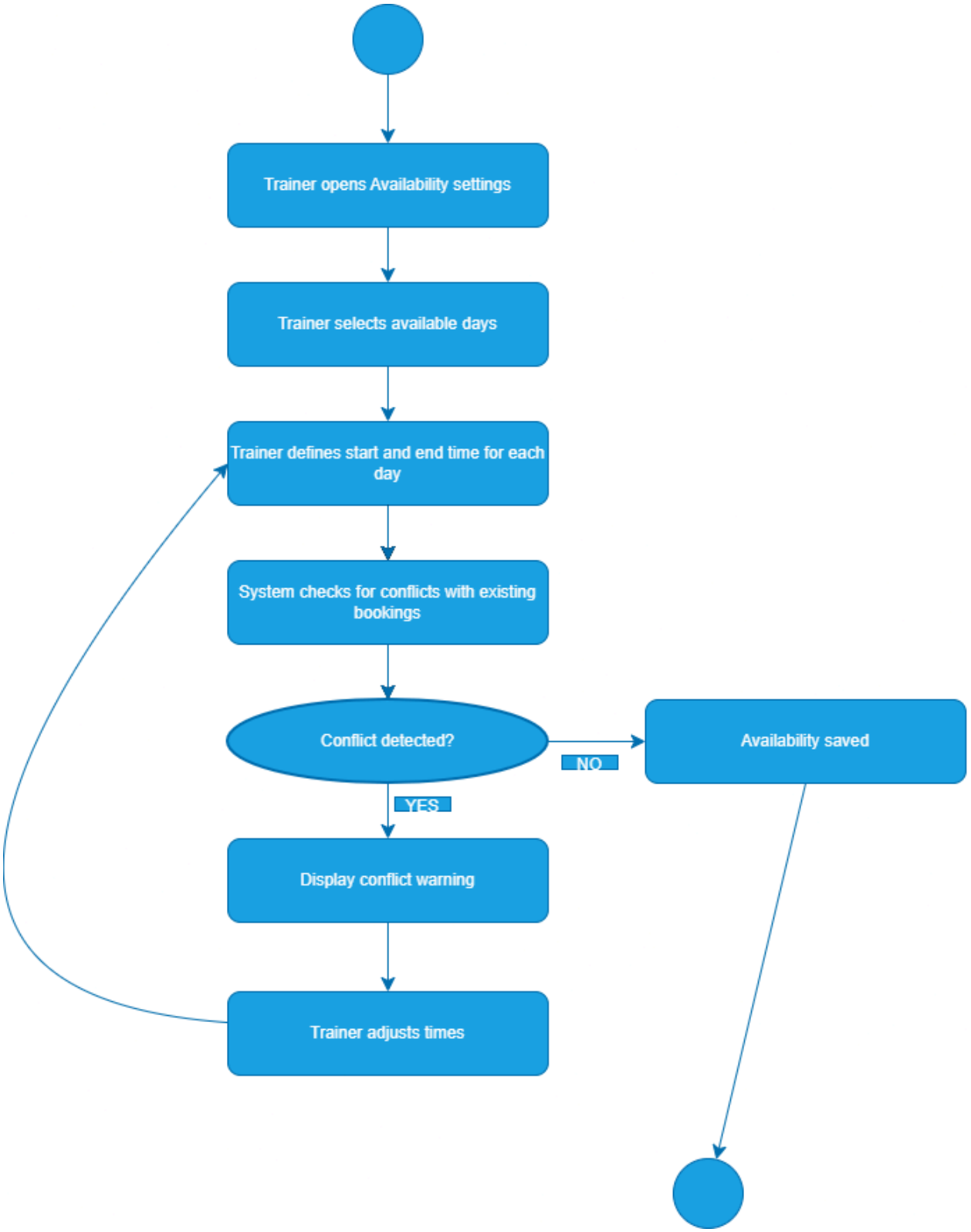



Figure 2: Activity Diagram - Trainer Sets Availability

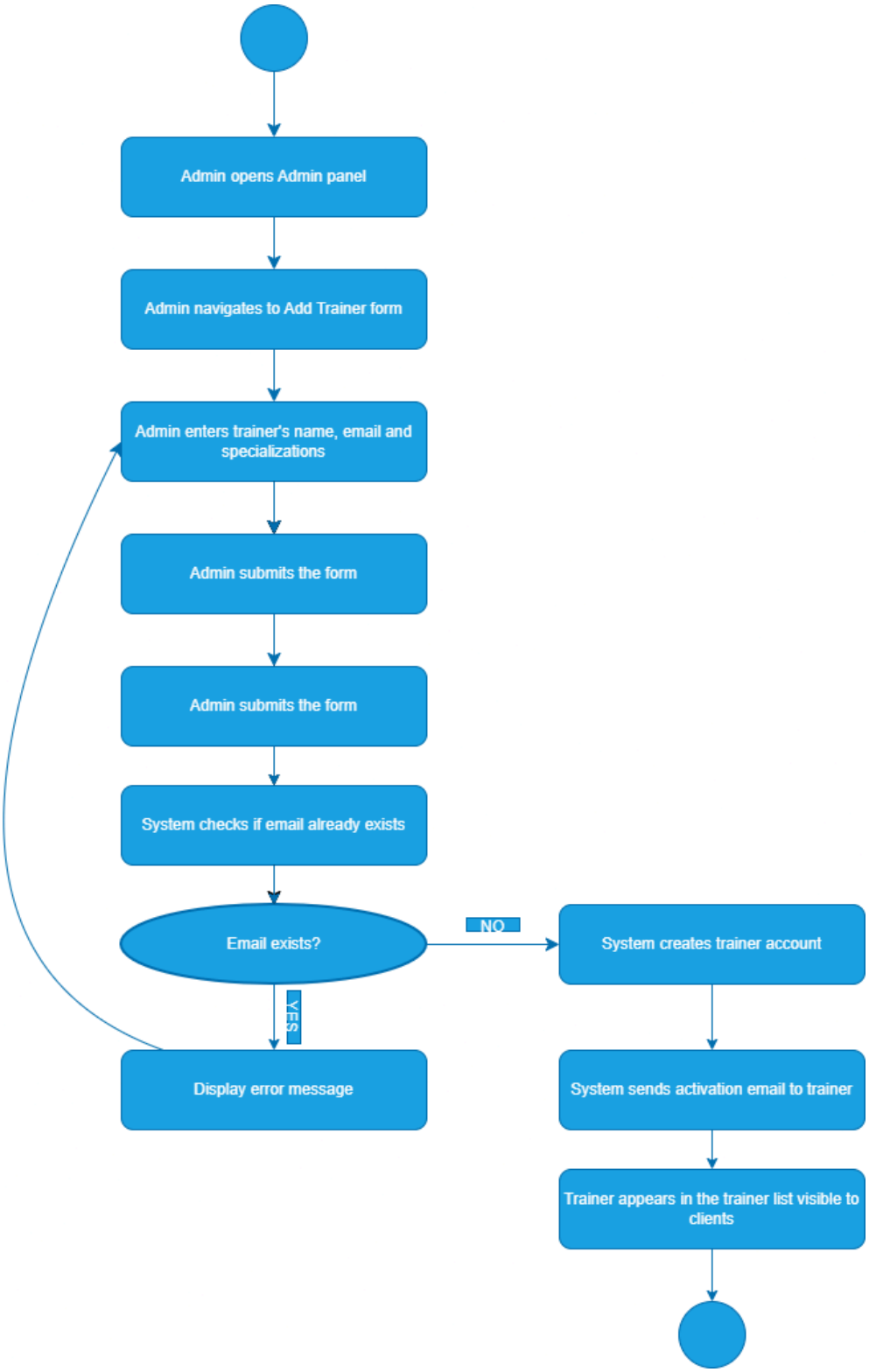



Figure 3: Activity Diagram - Admin Adds a New Trainer

## 4.2 Sequence Diagrams

The following sequence diagrams illustrate the interactions between actors and system components over time for three core functionalities of the FitSchedule application. The first diagram covers the complete booking flow from the moment a client selects a date and time until the reservation is confirmed and the trainer is notified. The second diagram shows how a trainer updates their profile and specializations, which directly affects how clients discover them on the platform. The third diagram covers the admin flow for adding a new trainer, including the system-side account creation and email activation process. Each diagram includes at least one fragment operator to represent conditional or optional behaviour within the flow.

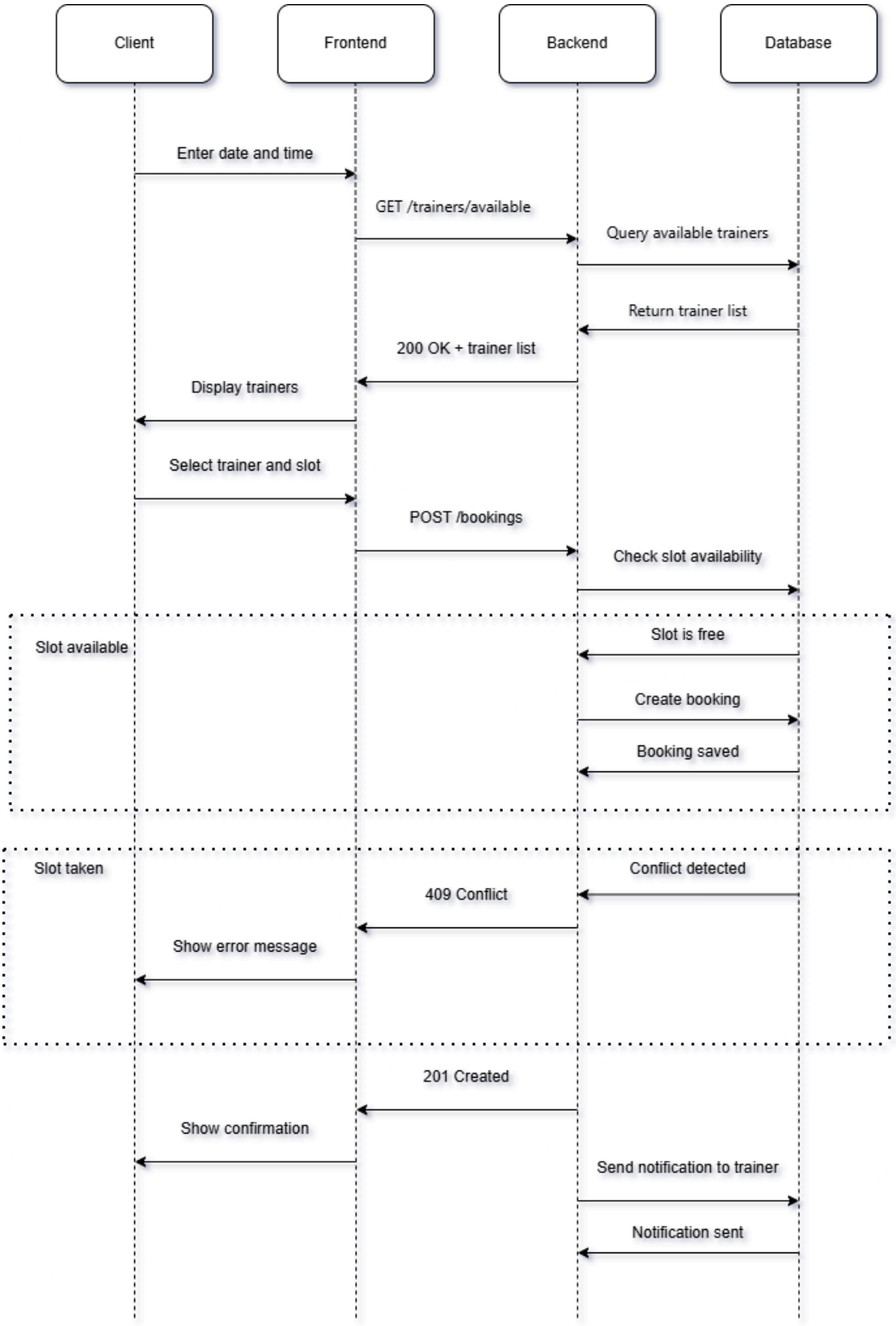



Figure 4: Sequence Diagram - Search & Book a Trainer

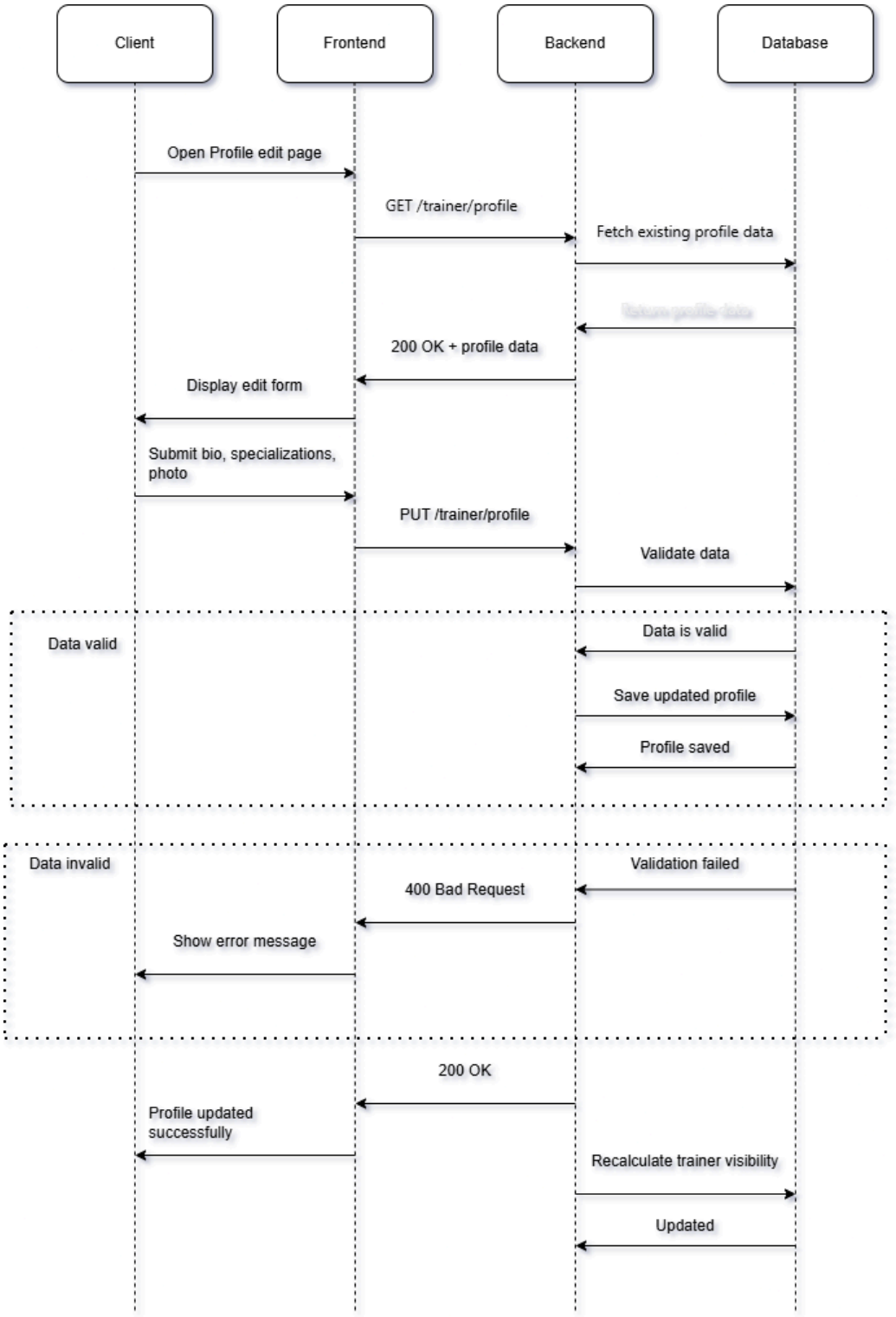



Figure 5: Sequence Diagram - Trainer Sets Up Profile and Specializations

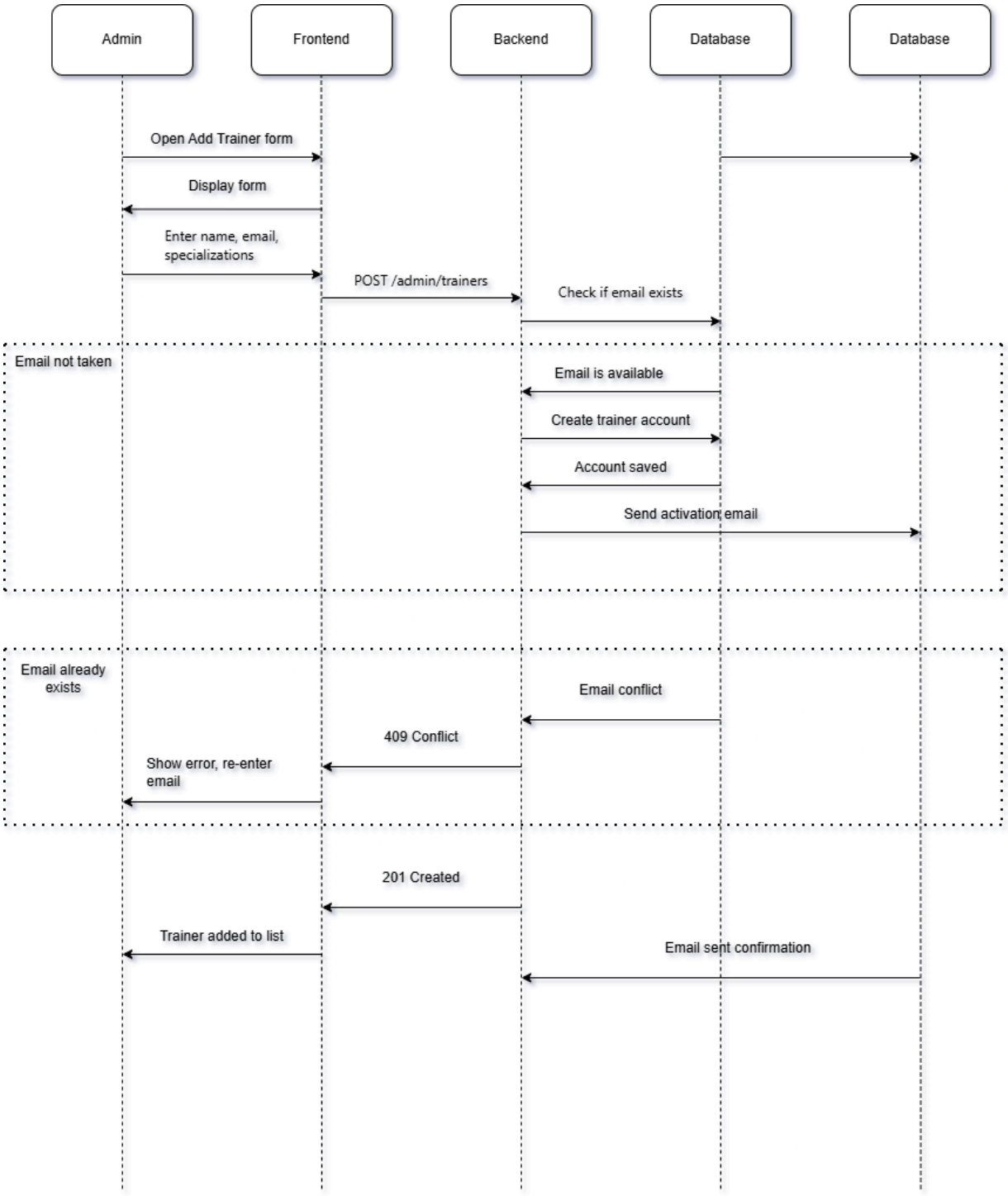



Figure 6: Sequence Diagram  - Admin Adds a New Trainer

## 4.3 Class Diagram

The following class diagram presents the structure of the FitSchedule application, showing all major entities, their attributes and methods, and the relationships between them. The diagram includes seven classes: User, Client, Trainer, Booking, TimeSlot, Availability, and Review.

Client and Trainer both extend the User class through inheritance, sharing common attributes such as first name, last name, email, and password hash, while each adding their own role-specific attributes and methods. Trainer is connected to Availability through a composition relationship, meaning that availability records cannot exist without a trainer. Trainer is also connected to TimeSlot through an aggregation relationship, as time slots are generated from availability but can exist independently. A Client makes zero or more Bookings, and a Trainer receives zero or more Bookings. Each Booking reserves exactly one TimeSlot. A Client can write zero or more Reviews, each of which is associated with a completed Booking.

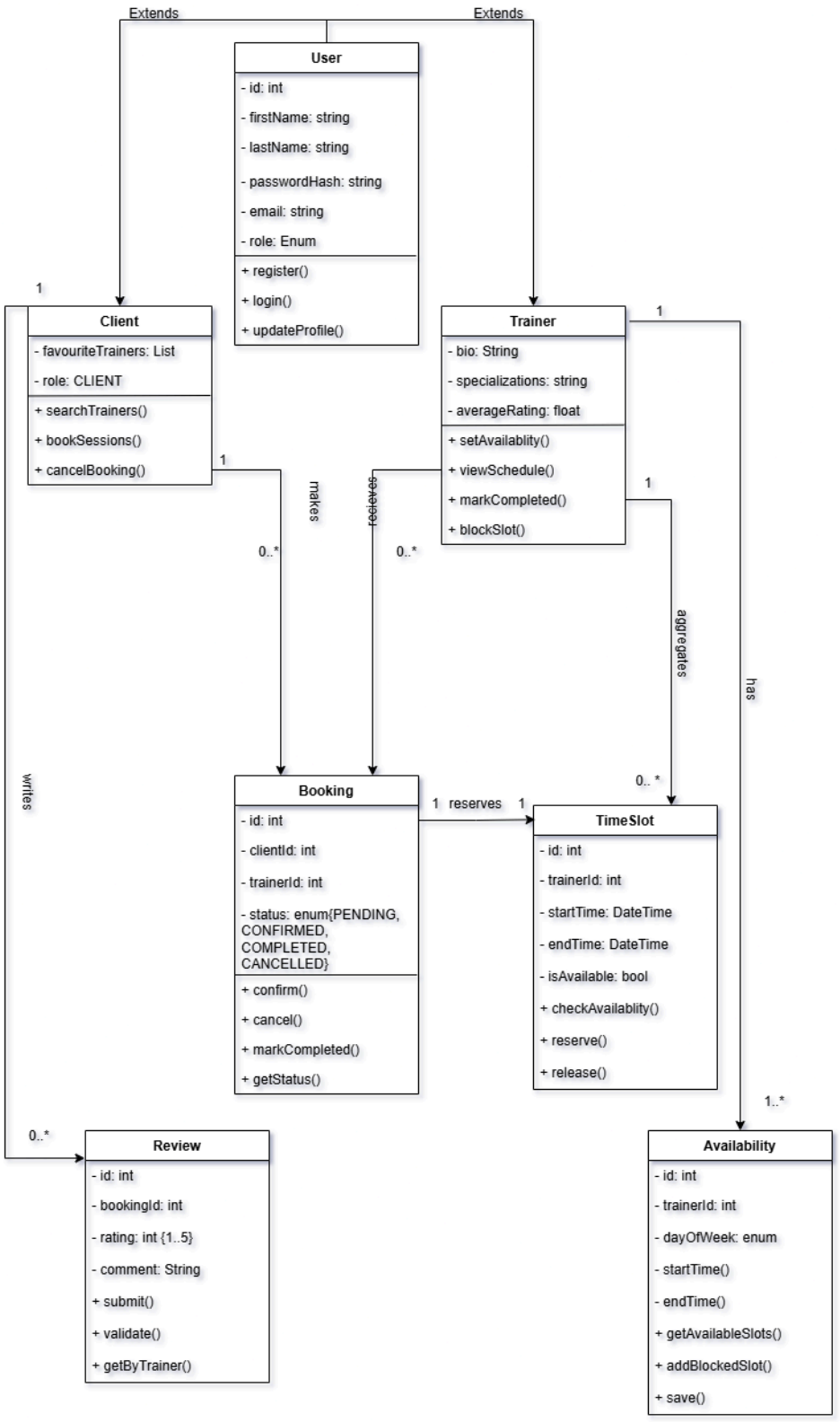



Figure 7: Class Diagram - FitSchedule

## 2. Project Structure (Milestone 2)

### 2.1. Technologies

FitSchedule[2] was developed as a full-stack web application using modern technologies for backend development, frontend implementation, database management, security, and API documentation.

For backend development, the project uses **Java 21** as the primary programming language. Java 21 provides modern language features, performance optimizations, and long-term support, making it suitable for scalable enterprise applications.

The backend architecture is built using the **Spring Boot** framework. Spring Boot was chosen because it simplifies application setup and configuration, reduces boilerplate code, and enables rapid development of RESTful services. Within Spring Boot, several integrated modules were used:

- **Spring Web** for building REST APIs and handling HTTP requests and responses.
- **Spring Data JPA** for database communication and object-relational mapping (ORM), using Hibernate as the persistence provider.
- **Spring Validation** for validating incoming request data using annotations such as @Valid, @NotNull, and @Email.
- **Spring Security** for authentication and authorization mechanisms.
- **Spring DevTools** for improving development productivity through automatic application restart.

For authentication and authorization, the system uses **JSON Web Token (JWT)** implemented through the **JJWT library**. JWT is used for secure user authentication and session management by generating and validating access tokens.

For database management, the application uses **MySQL** as the primary relational database. MySQL was selected because of its reliability, performance, and compatibility with Spring Data JPA. Database schema versioning and migrations are handled using **Flyway**, which ensures controlled and consistent database updates across development environments.

For frontend development, the project uses **React**, a component-based JavaScript library for building dynamic and responsive user interfaces. React enables modular frontend architecture and efficient state management.

For API testing and documentation, **Swagger (OpenAPI)** was integrated using SpringDoc OpenAPI. Swagger provides automatically generated API documentation and allows testing API endpoints directly through the browser.

Several third-party development tools were also used during implementation:

- **IntelliJ IDEA** as the primary IDE for backend development.
- **MySQL Workbench** for database design, administration, and query execution.

[2] GitHub link: https://github.com/sparah27/FitSchedule

- **WampServer** for local server environment management, including Apache and MySQL services.
- **Apache HTTP Server** as part of the local server infrastructure.
- **Apache Maven** for dependency management, project build lifecycle, and plugin execution.
- **Project Lombok** for reducing boilerplate code by generating getters, setters, constructors, and builders automatically.

For testing, the project uses **Spring Boot Test** and **Spring Security Test** for unit and integration testing of application logic and security configurations.

This technology stack provides FitSchedule with a scalable, secure, maintainable, and efficient architecture suitable for managing personal training bookings and user scheduling operations.

## 2.2. Database Entities

Provide a list of tables or entities you have in your database/schema and a ER digram as an image. Make sure you have at least 3 entities. If it is not obvious from the name of the table/entity what it is used for, also provide a brief explanation next to it. For example:

1. **users** — stores both clients and trainers in a single table
2. **availabilities** — defines a trainer's recurring weekly schedule (e.g. "available on Mondays 09:00–11:00"), used to indicate general availability before specific slots are created.
3. **time_slots** — represents specific bookable datetime windows generated from a trainer's availability
4. **bookings** — the core transaction table. Links a client, a trainer, and a time slot together, and tracks the booking lifecycle
5. **notifications** — stores in-app notifications sent to any user.

# 3.Coding standards

## 3.1 Coding Standards

**Backend**

Package structure: com.fitschedule.fitschedule.app.{controller, service, repository, model, dto, config, security, exception}

Class names: PascalCase (BookingService, TrainerController, JwtAuthenticationFilter)

Method names: camelCase, verb-first (createBooking, cancelBooking, getMyUpcomingBookings)

Constants and enums: UPPER_SNAKE_CASE (BookingStatus.CONFIRMED, TimeSlotStatus.AVAILABLE)

Lombok annotations used throughout: @RequiredArgsConstructor, @Builder, @Data, @Getter, @Setter to eliminate boilerplate

All public service methods are annotated with @Transactional or @Transactional(readOnly = true)

Input validation via Bean Validation (@Valid on controller parameters, @NotBlank/@Email/@NotNull on DTOs)

OpenAPI annotations (@Tag, @Operation, @Parameter, @SecurityRequirement) on all controllers for auto-generated Swagger UI

No magic strings in business logic; domain values represented as enums

**Frontend**

Component files: PascalCase.jsx (TrainerProfile.jsx, BookingModal.jsx)

Variables and functions: camelCase (handleSubmit, fetchTrainers, isLoading)

Service files: camelCase (bookingService.js, authService.js)

Styling: Tailwind CSS utility classes exclusively; no inline styles or custom CSS files except App.css for root-level layout

State: React hooks (useState, useEffect) for local state; Zustand for global auth state

Async/await used consistently throughout; no raw .then() chains

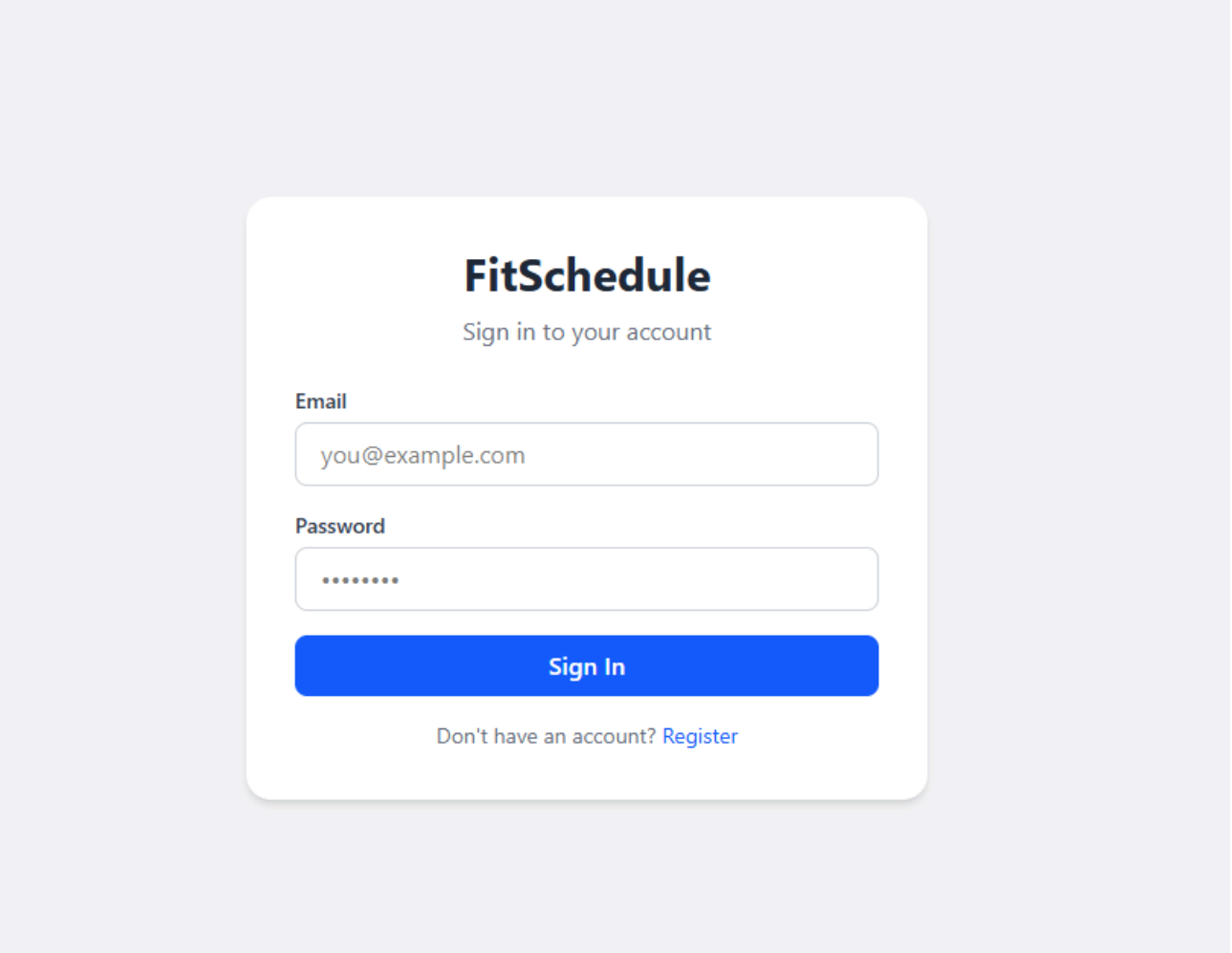
ESLint + Prettier enforced via eslint.config.js and .prettierrc for code formatting consistency

Environment variables in .env file (VITE_ prefix for Vite exposure)

## 3.2 Project Functionalities and Screenshots

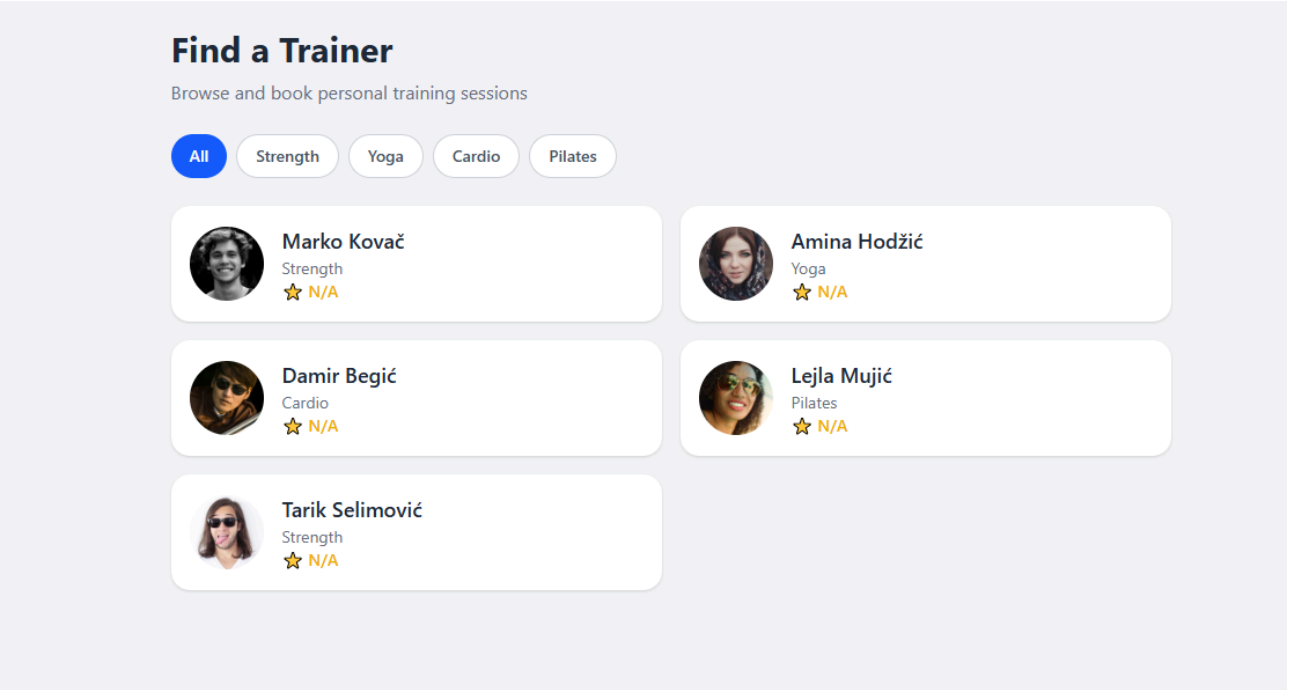
### User Authentication

New users register as clients by providing their first name, last name, email, password, and optionally phone number. Passwords are hashed with BCrypt before storage. On login, the backend validates credentials and returns a JWT token valid for 24 hours. The frontend stores this token in Zustand and attaches it as a Bearer header to all subsequent requests via an axios interceptor.



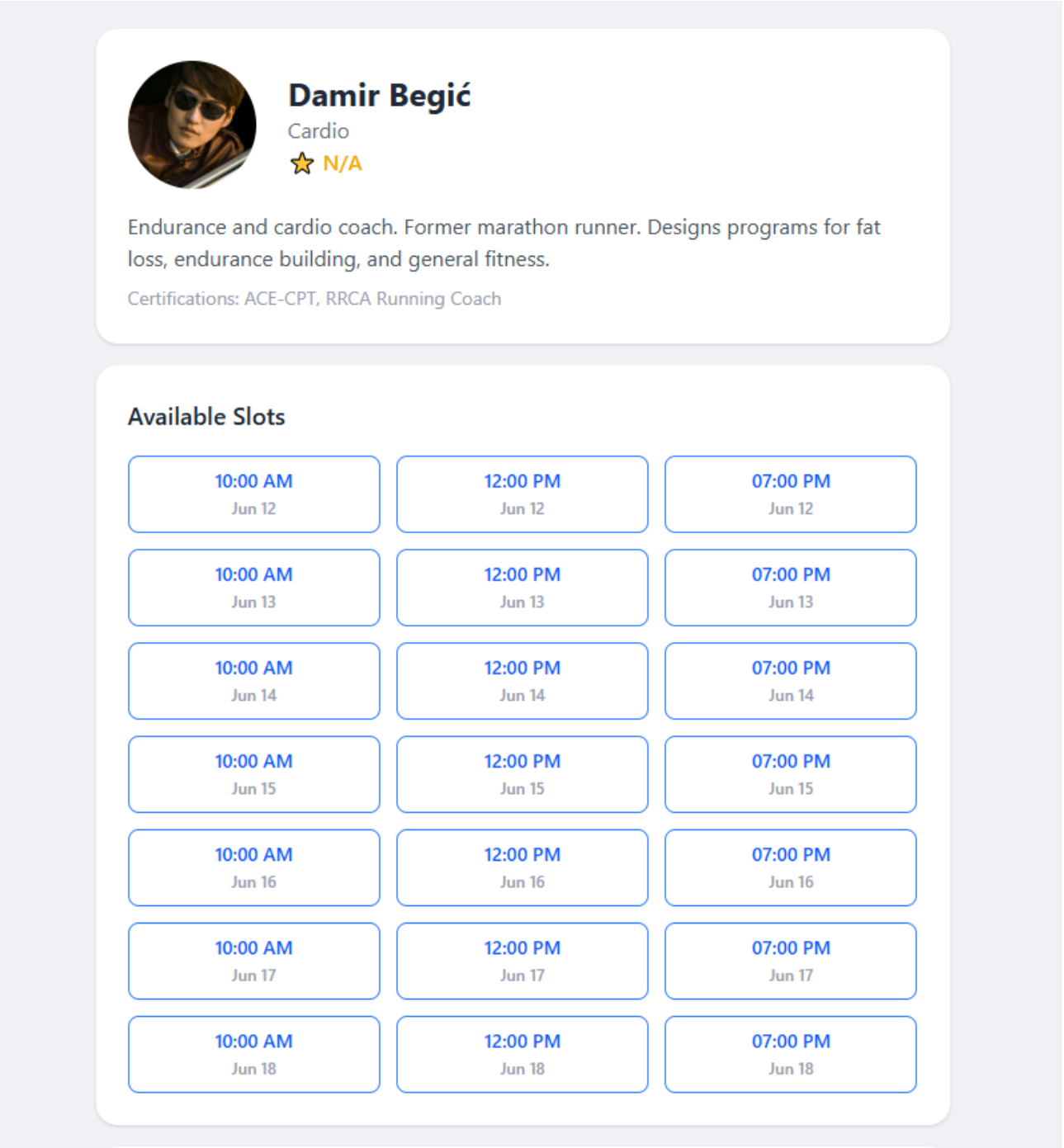

### Trainer Browsing & Search

The Trainers page displays all active trainers as cards showing name, specialization, and a profile photo. Users can filter trainers by specialization. The Search page allows users to find trainers who have at least one available slot in a specified date/time range, enabling goal-driven discovery rather than manual calendar browsing.



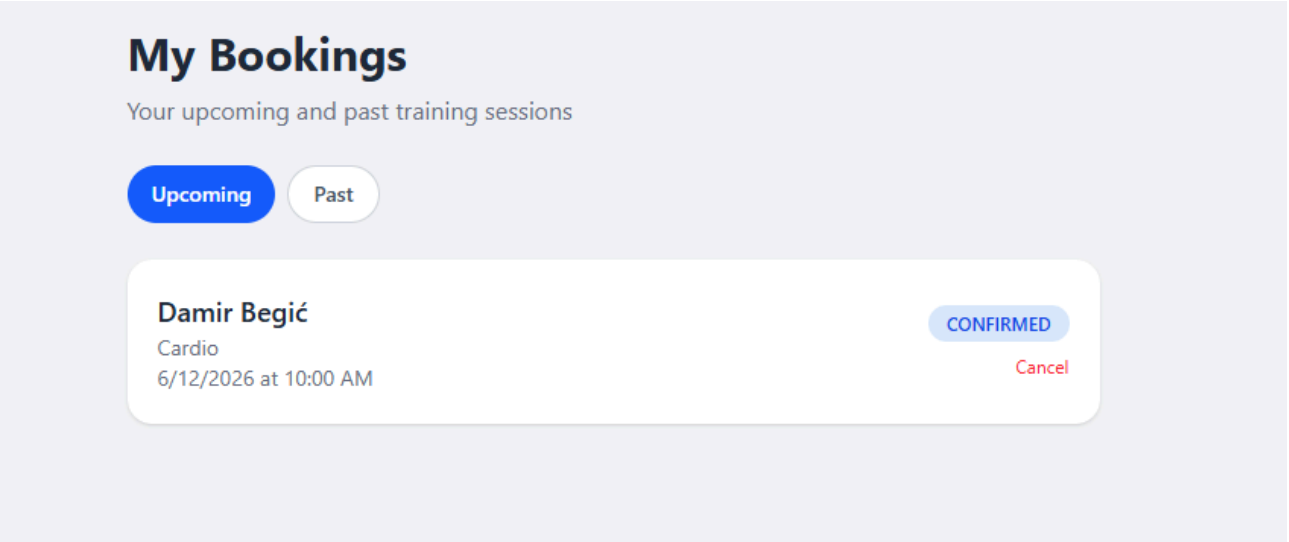

**Trainer Profile & Availability**

Each trainer has a detailed profile page showing their bio, certifications, specialization, and a list of available time slots for a chosen date range. Time slots are fetched from the backend and rendered in a scrollable list. Slots in the past or already booked are automatically excluded.



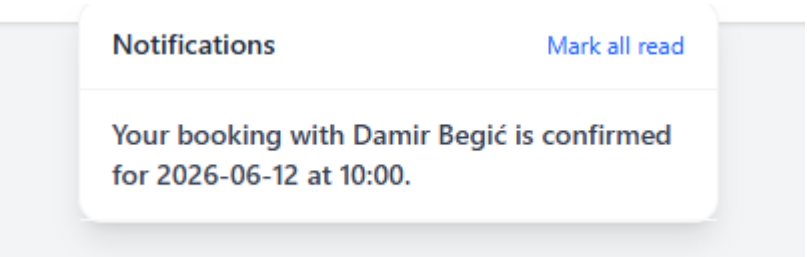

**Booking Management**

Clients book a session by selecting an available time slot on a trainer's profile page, which triggers a confirmation modal. The backend atomically marks the slot as BOOKED, creates a Booking record with status CONFIRMED, and dispatches notifications to both the client and trainer. Clients can view their upcoming and past bookings on the My Bookings page and cancel upcoming sessions (which atomically resets the slot to AVAILABLE and sends cancellation notifications).



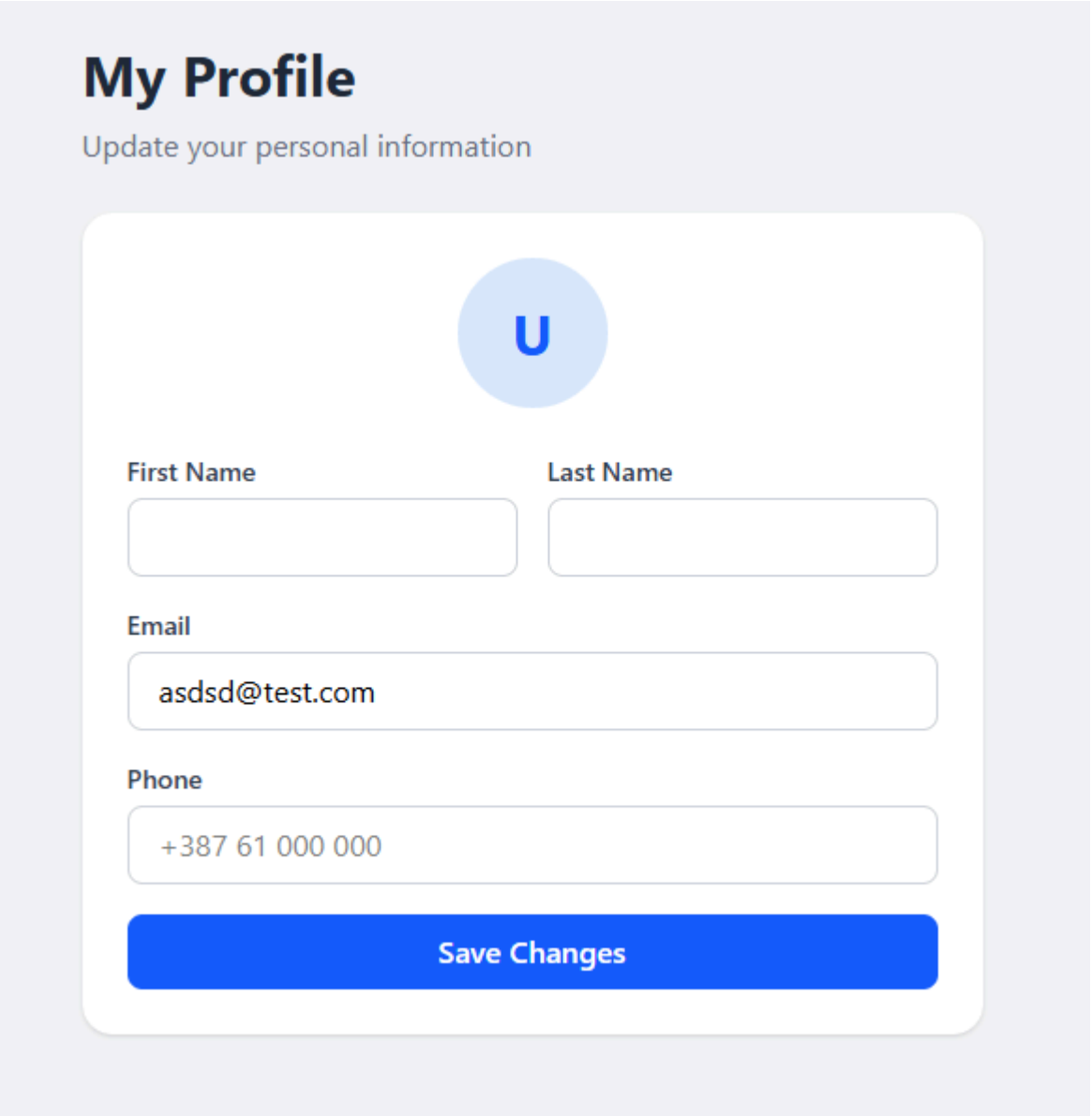

**Notifications**

The notification system creates in-app notifications for key booking events: booking confirmation (sent to both client and trainer), booking cancellation by client (sent to both parties), and booking cancellation by trainer. Unread notifications are highlighted; clients can mark them as read from the Navbar dropdown




**Profile Management**

Authenticated users can view and update their profile information (first name, last name, phone number) from the Profile page. The form pre-populates with the current user's data fetched from the backend.




## 3.3 Tests

### Test Strategy

Testing follows a layered approach targeting the most critical business logic components: service-layer unit tests covering booking creation and cancellation rules, and application context integration tests verifying that the Spring Boot context loads correctly with all beans wired.

## Unit Tests — BookingService

**createBooking — happy path**

Verifies that a valid client booking on an AVAILABLE future slot creates a CONFIRMED Booking, sets the slot to BOOKED, and triggers two notifications (client + trainer). Mocks: UserRepository, TimeSlotRepository, BookingRepository, NotificationService.

**createBooking — slot not available**

Asserts that attempting to book an already BOOKED slot throws SlotNotAvailableException and that bookingRepository.save() is never called.

**createBooking — past slot**

Asserts that booking a slot whose startAt is in the past throws SlotNotAvailableException.

**createBooking — non-client user**

Asserts that a Trainer entity calling createBooking throws ForbiddenActionException.

**cancelBooking — happy path**

Verifies that cancelling a CONFIRMED future booking sets status to CANCELLED, frees the slot (AVAILABLE), sets cancelledAt, and triggers two notifications.

**cancelBooking — already cancelled**

Asserts that cancelling a booking not in CONFIRMED status throws SlotNotAvailableException.

**cancelBooking — past session**

Asserts that cancelling a booking whose session has already started or passed throws SlotNotAvailableException.

**cancelBooking — wrong user**

Asserts that a user who does not own the booking throws ForbiddenActionException

## 3.4 Individual Contributions

**Adi Dzumhur**

Designed and implemented the complete React frontend using Vite, Tailwind CSS 4, and React Router v7
Built all page components: Login, Register, Trainers, TrainerProfile, MyBookings, Search, Profile
Implemented global authentication state management using Zustand (authStore.js)
Created the axios API service layer (api.js) with JWT Bearer token injection and all service modules
Designed the booking modal (BookingModal.jsx) and the ProtectedRoute component
Built the Navbar component with notification dropdown integration
Configured ESLint, Prettier, and the Vite build pipeline
Wrote and maintained frontend .env configuration and environment setup documentation

**Sarah Spahic**

Designed the relational database schema (5 tables: users, availabilities, time_slots, bookings, notifications) using Single Table Inheritance
Wrote all Flyway migration scripts (V1 schema, V2/V3/V4 seed data for trainers and time slots)
Implemented the full Spring Boot backend: entities, repositories, services, controllers, DTOs
Built JWT-based stateless authentication (JwtService, JwtAuthenticationFilter, CustomUserDetailsService)
Configured Spring Security filter chain with CORS, CSRF disabled, and per-route authorization rules
Implemented the NotificationService for automated in-app notifications on booking events
Set up OpenAPI/Swagger documentation with springdoc-openapi and annotated all controllers
Wrote unit and integration tests for BookingService (8 test cases) and ApplicationTests